CSE-2204 Algorithms

Complete Lab Final Exam Guide

Graph Traversal • Shortest Path Algorithms • Dynamic Programming • Divide & Conquer • Recursion / Permutation / Combination / Path-Enumeration

28 fully-verified C++ solution files, each following the fixed muhtasim() lab template, bundling the core algorithm plus curated popular/interview LeetCode variants (cross-referenced against kamyu104/LeetCode-Solutions) and every exact instructor problem sample, verified byte-for-byte.

Fixed Code Template

Every .cpp file in this package uses this exact skeleton. Helper functions/classes may be added above muhtasim(), but the three top lines and the entire main() are identical in every file:

```
#include<bits/stdc++.h>
#define ll long long
using namespace std;
void muhtasim()
{
// topic-specific logic here
}
int main()
{
ios::sync_with_stdio(false);
cin.tie(nullptr);
ll tests;
cin>>tests;
while(tests--)
{
muhtasim();
}
}
```

muhtasim() reads one dataset's input per test case and prints that dataset's answer(s). For topics with an exact instructor-issued I/O spec, muhtasim() solves that spec precisely (verified byte-for-byte against the given samples). Every additional bundled LeetCode problem appears as a clearly banner-commented section using its own small demo input, in the same run.

Dynamic Programming vs. Divide & Conquer vs. Greedy

Property	Divide & Conquer	Dynamic Programming	Greedy
Subproblems	Independent, non-overlapping	Overlapping (reused)	N/A - one pass, no subproblems
Combine step	Merge independent solutions	Combine via recurrence + memo/table	Pick locally optimal choice
Optimal substructure	Yes	Yes	Yes (for problems where greedy is valid)
Typical complexity	O(n log n) (e.g. merge sort)	O(n^2), O(n*W), etc.	O(n log n) or O(n)
Examples	Merge sort, closest pair, quicksort	Knapsack, LCS, LIS, MCM, rod cutting	Activity selection, Huffman, Dijkstra (proof-based)
When it fails	-	Without memoization: exponential blowup	Fails when local optimum != global optimum

Graph Traversal

Graph Representation

File: Graph/graph_representation.cpp **Complexity:** O(N+M) list build, O(N^2) matrix build/space

Build and convert between adjacency list and adjacency matrix representations; weighted/directed variants.

Bundled LeetCode / interview problems: LC 133 Clone Graph; LC 997 Find the Town Judge; LC 1042 Flower Planting With No Adjacent; LC 1557 Minimum Number of Vertices to Reach All Nodes

Breadth-First Search (BFS)

File: Graph/bfs.cpp **Complexity:** O(N+M)

Level-order traversal from a source; gives shortest hop-distance in unweighted graphs.

Bundled LeetCode / interview problems: LC 200 Number of Islands (BFS); LC 994 Rotting Oranges; LC 1091 Shortest Path in Binary Matrix; LC 127 Word Ladder; LC 785 Is Graph Bipartite; LC 542 01 Matrix

Depth-First Search (DFS)

File: Graph/dfs.cpp **Complexity:** O(N+M)

Recursive/stack-based traversal; discovery/finish times and edge classification (Tree/Back/Forward/Cross).

Instructor: DFS Again (discovery/finish) + Edge Classification

```
Input: 6\n6\n1 2\n1 3\n0 4\n2 3\n5 4\n5 0
Output: 0: 1 6\n1: 7 12\n2: 8 11\n3: 9 10\n4: 2 5\n5: 3 4
```

Bundled LeetCode / interview problems: LC 200 Number of Islands; LC 130 Surrounded Regions; LC 547 Number of Provinces; LC 1319 Make Network Connected; LC 261 Graph Valid Tree

Topological Sort

File: Graph/topological_sort.cpp **Complexity:** O(N+M)

Linear ordering of DAG vertices respecting all directed edges; via DFS-finish-order or Kahn's BFS.

Instructor: Topological Order

```
Input: 6\n5\n1 3\n1 2\n0 1\n3 4\n2 5
Output: 0\n1\n2\n5\n3\n4
```

Bundled LeetCode / interview problems: LC 207/210 Course Schedule I/II; LC 269 Alien Dictionary; LC 802 Find Eventual Safe States; LC 1136 Parallel Courses; All valid topological orderings (backtracking)

Strongly Connected Components

File: Graph/scc.cpp **Complexity:** O(N+M)

Maximal sets of mutually reachable vertices in a directed graph; Kosaraju's (2-pass DFS) and Tarjan's (1-pass low-link).

Instructor: SCC (Kosaraju's)

```
Input: 8\n8\n1 3\n0 1\n3 0\n3 7\n7 3\n5 6\n6 4\n6 5
Output: 6 5\n4\n2\n1 7 3 0
```

SCC partition is unique but in-component print order is not canonical; this file's verified output has the same 4 groups {5,6} {4} {2} {0,1,3,7} as the sample.

Bundled LeetCode / interview problems: LC 1557 Min Vertices to Reach All Nodes; LC 2360 Longest Cycle in a Graph; Condensation graph construction

Euler Path / Circuit

File: Graph/euler_path.cpp **Complexity:** O(N+M)

A walk using every edge exactly once. Circuit iff every vertex has even degree (undirected) / in-deg=out-deg (directed); found via Hierholzer's algorithm.

Bundled LeetCode / interview problems: LC 332 Reconstruct Itinerary; LC 753 Cracking the Safe (De Bruijn); LC 2097 Valid Arrangement of Pairs

Articulation Points

File: Graph/articulation_points.cpp **Complexity:** O(N+M)

Vertices whose removal disconnects the graph; found via Tarjan's DFS low-link values.

Instructor: Articulation Points

```
Input: 6\n5\n1 3\n1 2\n0 1\n3 4\n2 5
Output: 1\n2\n3
```

Bundled LeetCode / interview problems: LC 1192 Critical Connections (shared low-link machinery)

Bridges

File: Graph/bridges.cpp **Complexity:** O(N+M)

Edges whose removal disconnects the graph; low[v] > disc[u] on a DFS tree edge.

Instructor: Bridges

```
Input: 6\n6\n1 3\n1 2\n2 3\n0 1\n3 4\n5 0
Output: 0 1\n0 5\n3 4
```

Bundled LeetCode / interview problems: LC 1192 Critical Connections in a Network

Bi-connected Components

File: Graph/biconnected_components.cpp **Complexity:** O(N+M)

Maximal subgraphs with no articulation point; found via an edge-stack popped at each low[v]>=disc[u] event. Rare as a direct LeetCode problem (GfG/CLRS staple).

Bundled LeetCode / interview problems: Cross-referenced with articulation points from the same DFS pass

Shortest Path Algorithms

Dijkstra's Algorithm

File: ShortestPath/dijkstra.cpp **Complexity:** O((N+M) log N)
Single-source shortest paths on non-negative weighted graphs via a min-priority queue.
Instructor: A. Dijkstra (distances) + B. Path of Dijkstra (lexicographically smallest path)

```
Input: A: 7\n6\n1 3 2\n1 2 5\n0 1 3\n3 2 1\n3 4 50\n2 5 10 | B: 7\n6\n4 3 2\n4 2 5\n0 4 3\n3 2 1\n3 6 50\n2 1 10
Output: A: 0: 0 / 1: 3 / 2: 6 / 3: 5 / 4: 55 / 5: 16 / 6: Infinity | B: 0 / 4 / 3 / 2 / 1
```

Bundled LeetCode / interview problems: LC 743 Network Delay Time; LC 1514 Path with Maximum Probability; LC 1631 Path With Minimum Effort; LC 1976 Number of Ways to Arrive at Destination

Bellman-Ford + Negative Cycle Detection

File: ShortestPath/bellman_ford.cpp **Complexity:** O(N*M)
Relax all edges N-1 times; an Nth relaxation that still improves a distance signals a negative cycle.
Bundled LeetCode / interview problems: LC 787 Cheapest Flights Within K Stops

Shortest Path in a DAG

File: ShortestPath/dag_shortest_path.cpp **Complexity:** O(N+M)
Topological order + single DP relaxation pass gives both shortest and longest path in O(N+M), no queue needed.
Bundled LeetCode / interview problems: LC 1786 Number of Restricted Paths; LC 329 Longest Increasing Path in a Matrix; Critical-path scheduling

Floyd-Warshall

File: ShortestPath/floyd_warshall.cpp **Complexity:** O(N^3)
All-pairs shortest paths via triple-loop DP over intermediate vertices; naturally handles negative edges (not negative cycles) and reconstructs paths via a next[][] table.
Bundled LeetCode / interview problems: LC 1334 Find the City With Smallest Number of Neighbors; LC 1462 Course Schedule IV; LC 2101 Detonate the Maximum Bombs

Johnson's Algorithm

File: ShortestPath/johnsons_algorithm.cpp **Complexity:** O(N*M log N)
All-pairs shortest paths for sparse graphs with negative edges: Bellman-Ford reweights edges to be non-negative, then Dijkstra runs from every source.
Bundled LeetCode / interview problems: Rare as a direct LeetCode problem; cross-verified against a Floyd-Warshall run on the same graph

Dynamic Programming

Rod Cutting

File: DP/rod_cutting.cpp **Complexity:** O(n^2)
Classic CLRS DP: maximize revenue cutting a rod into integer-length pieces with given prices.
Bundled LeetCode / interview problems: LC 343 Integer Break (closest LC analogue)

General DP & Memoization

File: DP/dp_intro_memoization.cpp **Complexity:** Exponential (naive) -> O(n) (memoized)
Naive recursion vs. top-down memoization vs. bottom-up tabulation, with a real timing comparison.
Bundled LeetCode / interview problems: LC 509 Fibonacci Number; LC 70 Climbing Stairs; LC 62 Unique Paths

Coin-Related Problems

File: DP/coin_problems.cpp **Complexity:** O(n*amount)
Minimum coins for a target (unbounded knapsack style) and number of ways to make change (combinations vs. permutations of coin order).
Bundled LeetCode / interview problems: LC 322 Coin Change; LC 518 Coin Change II; LC 377 Combination Sum IV; LC 279 Perfect Squares

Longest Increasing Subsequence (LIS)

File: DP/lis.cpp **Complexity:** O(n^2) or O(n log n)
Longest strictly increasing subsequence; O(n^2) DP and O(n log n) patience-sorting/binary-search method.
Bundled LeetCode / interview problems: LC 300 LIS; LC 673 Number of LIS; LC 354 Russian Doll Envelopes

Longest Common Subsequence (LCS)

File: DP/lcs.cpp **Complexity:** O(n*m)
Longest subsequence common to two (or three) strings; foundation for edit distance and diff tools.
Bundled LeetCode / interview problems: LC 1143 LCS; LC 1092 Shortest Common Supersequence; LC 72 Edit Distance; LC 115 Distinct Subsequences

0/1 Knapsack

File: DP/knapsack_01.cpp **Complexity:** O(n*W) memoized vs O(2^n) naive
Choose items with weight/value to maximize value under a capacity constraint, each item used at most once. Includes the graded assignment: naive recursion vs. memoized DP, timed on 10+ cases (5-35 items), proving memoization's speedup (measured up to ~27x at n=25 in this package's own test run).
Bundled LeetCode / interview problems: LC 416 Partition Equal Subset Sum; LC 494 Target Sum; LC 1049 Last Stone Weight II; LC 474 Ones and Zeroes

Matrix Chain Multiplication

File: DP/matrix_chain_multiplication.cpp **Complexity:** O(n^3)
Minimize scalar multiplications when multiplying a chain of matrices via optimal parenthesization; interval DP.
Bundled LeetCode / interview problems: LC 312 Burst Balloons; LC 1000 Minimum Cost to Merge Stones; LC 1130 Min Cost Tree From Leaf Values

Rock Climbing (Min-Cost Climbing / Frog Jump)

File: DP/rock_climbing.cpp **Complexity:** O(n)

Minimum cost to reach the top of a staircase, or a frog jumping across stones; simple 1D DP with a jump-set generalization.

Bundled LeetCode / interview problems: LC 746 Min Cost Climbing Stairs; LC 70 Climbing Stairs; LC 55/45 Jump Game I/II; LC 403 Frog Jump

Applications of DP: Path Counting

File: DP/dp_applications_paths.cpp **Complexity:** O(rows*cols)

Counting and enumerating distinct paths/ways through a grid: unique paths, obstacles, min path sum, and actual path enumeration.

Bundled LeetCode / interview problems: LC 62 Unique Paths; LC 63 Unique Paths II; LC 64 Minimum Path Sum; LC 174 Dungeon Game; LC 980 Unique Paths III (enumerate)

Divide & Conquer

Counting Inversions via Merge Sort

File: DivideConquer/counting_inversions.cpp **Complexity:** O(n log n)
Count pairs (ia[j]) in O(n log n) by counting cross-inversions during merge-sort's merge step.
Bundled LeetCode / interview problems: LC 493 Reverse Pairs; LC 315 Count of Smaller Numbers After Self

Closest Pair of Points

File: DivideConquer/closest_pair_points.cpp **Complexity:** O(n log n)
Find the minimum-distance pair among n 2D points by recursively splitting on x, then checking a thin strip near the split line.
Bundled LeetCode / interview problems: Classic CLRS/GfG problem; rare as a direct LeetCode problem

Recursion / Permutation / Combination / Path Enumeration

Constrained Permutations

File: Recursion_PermComb/permutations_all.cpp **Complexity:** Exponential (backtracking with pruning), N<=20

Generate all permutations of 1..N in lexicographic order subject to: no two evens adjacent, no two odds adjacent, adjacent difference < 5.

Instructor: Problem A – Permutation

```
Input: N=1 / N=2 / N=4
Output: N=1: 1 | N=2: 1 2 / 2 1 | N=4: 1 2 3 4 / 1 4 3 2 / 2 1 4 3 / 2 3 4 1 / 3 2 1 4 / 3 4 1 2 /
4 1 2 3 / 4 3 2 1
```

Bundled LeetCode / interview problems: LC 46 Permutations; LC 47 Permutations II; LC 31 Next Permutation; LC 60 Permutation Sequence; LC 526 Beautiful Arrangement

Combinations

File: Recursion_PermComb/combinations_all.cpp **Complexity:** Exponential (backtracking)

Generate all k-subsets and combination-sum style problems via backtracking.

Bundled LeetCode / interview problems: LC 77 Combinations; LC 39/40/216 Combination Sum I/II/III; LC 78/90 Subsets/Subsets II; LC 17 Letter Combinations of a Phone Number; LC 22 Generate Parentheses

All Distinct Paths / Counting-Ways Enumeration

File: Recursion_PermComb/all_paths_distinct_ways.cpp **Complexity:** Exponential (backtracking)

Enumerate (not just count) all paths/arrangements: grid paths, word search, N-Queens, IP restoration, palindrome partitioning.

Bundled LeetCode / interview problems: LC 797 All Paths From Source to Target; LC 79 Word Search; LC 51/52 N-Queens; LC 93 Restore IP Addresses; LC 131 Palindrome Partitioning